

Progetto di Reti Logiche

Prova Finale — Anno Accademico 2025/2026

Prof. Fornaciari, Prof. Reghenzani

Caliment Sara

Bueno Moran Jadixsa Jamileth

Indice

1	Introduzione	2
1.1	Esempi di funzionamento	2
2	Architettura	4
2.1	Diagramma della FSM	4
2.2	Descrizione degli stati	5
3	Risultati Sperimentali	8
3.1	Simulazioni Behavioral	8
3.2	Simulazioni Post-Synthesis Functional	15
3.3	Report di Sintesi	15
4	Conclusione	16

1 Introduzione

Il componente progettato implementa un gestore di una lista ordinata di task memorizzata su una RAM esterna a 8 bit. La lista mantiene i task sempre ordinati per valore di priorità crescente. In particolare, l'indirizzo 0 della memoria viene adibito come contatore, indicando il numero di task presenti. Ogni task è identificato da un campo ID_TASK a 6 bit (valori da 1 a 63, dove lo 0 indica lista vuota), e da campo PRIORITY a 2 bit (valori da 0 a 3, dove 0 è la priorità più alta e 3 è la più bassa), i quali vengono concatenati in un unico byte: i 6 bit più significativi contengono ID_TASK e i 2 meno significativi la PRIORITY.

Il gestore della lista implementa quattro operazioni:

- OP = 00 — Incrementa (decrementando il livello gerarchico) il valore del campo PRIORITY di 1 per ciascun task, senza alterare l'ordine della lista. Il valore satura a 11 (priorità minima).
- OP = 01 — Rimuove il primo task della lista e ne restituisce l>ID_TASK in uscita; tutti gli altri task vengono scalati di una posizione. Caso limite: se la lista è vuota, l'uscita è 0x000000.
- OP = 10 — Gestisce l'inserimento di un nuovo task mantenendo l'ordinamento della lista, ponendolo in coda a tutti i task con uguale PRIORITY.
- OP = 11 — Svuota la lista, scrivendo 0 all'indirizzo 0 e lasciando invariato il contenuto degli indirizzi successivi.

Nella progettazione del componente è stato utilizzato il protocollo di handshake START–DONE sincrono sul fronte di salita del clock. Per l'avvio delle operazioni, la logica esterna porta START a 1 e, per segnalare il suo completamento, il modulo risponde portando DONE a 1. Solo dopo che START torna a 0, il modulo può riportare DONE a 0 per prepararsi ad una nuova operazione. Il RESET è asincrono.

1.1 Esempi di funzionamento

Si consideri una lista inizialmente vuota con la seguente sequenza di operazioni.

Stato iniziale

Indirizzo	Task (8 bit)	ID_TASK	PRIORITY
0	0x00000000	—	—

Operazione OP = 10 — Inserimento di quattro task

- Insert: ID_TASK = 15, PRIORITY = 0
- Insert: ID_TASK = 31, PRIORITY = 3
- Insert: ID_TASK = 20, PRIORITY = 0
- Insert: ID_TASK = 2, PRIORITY = 1

Indirizzo	Task (8 bit)	ID_TASK	PRIORITY
0	0x00000100	—	—
1	0x00111100	15	0
2	0x01010000	20	0
3	0x00001001	2	1
4	0x01111111	31	3

Operazione OP = 00 — Decremento priorità

Indirizzo	Task (8 bit)	ID_TASK	PRIORITY
0	0x00000100	—	—
1	0x00111101	15	1
2	0x01010001	20	1
3	0x00001010	2	2
4	0x01111111	31	3

Operazione OP = 01 — Estrazione (ID_TASK = 15)

Indirizzo	Task (8 bit)	ID_TASK	PRIORITY
0	0x00000011	—	—
1	0x01010001	20	1
2	0x00001010	2	2
3	0x01111111	31	3

Operazione OP = 11 — Svuotamento lista

Indirizzo	Task (8 bit)	ID_TASK	PRIORITY
0	0x00000000	—	—
1	0x01010001	20	1
2	0x00001010	2	2
3	0x01111111	31	3

Il contenuto degli indirizzi successivi rimane invariato (non è rilevante per il funzionamento).

2 Architettura

Il componente è stato implementato con un'architettura a due processi separati, il motivo di tale separazione è per una maggiore facilità di lettura, debugging e manutenibilità del codice, inoltre, porre assieme nello stesso process due logiche diverse di sviluppo può causare latch indesiderati. Di seguito l'architettura viene illustrata nella figura 1.

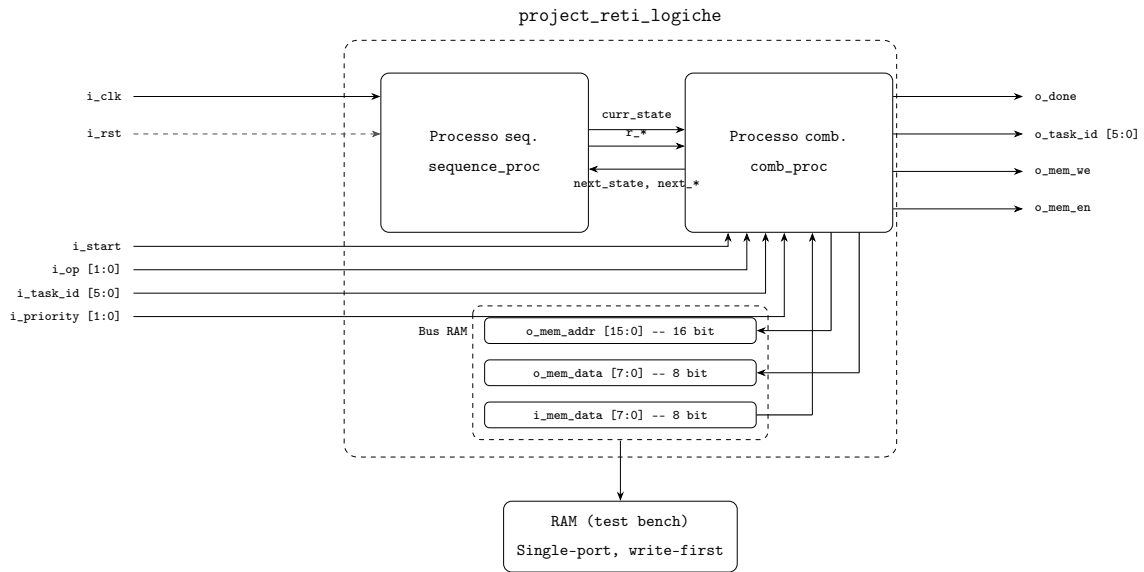


Figura 1: Schema architetturale del modulo `project_reti_logiche`

Il **processo sequenziale** (`sequence_proc`) gestisce il reset asincrono tramite `i_rst` e si occupa dell'aggiornamento e memorizzazione sincrono dello stato corrente e dei registri interni al *rising edge* del clock (`r_op`, `r_task_count`, `r_index`, `r_first`, `r_insert_pos`).

Mentre, il **processo combinatorio** (`comb_proc`), che è la vera e propria macchina a stati, determina in base allo stato corrente e agli ingressi: le transizioni tra gli stati, i valori dei registri al ciclo successivo (`next_*`), rispettando il protocollo handshake START–DONE, e i segnali di uscita verso la memoria (`o_mem_en`, `o_mem_we`, `o_mem_addr`, `o_mem_data`).

Ad ogni ciclo di clock, i seguenti segnali vengono messi a '0' dal default del processo combinatorio per rendere il comportamento prevedibile e ridurre il rischio di latch indesiderati: `o_done`, `o_mem_we`, `o_mem_en`, `o_mem_addr`, `o_mem_data`, `o_task_id`.

2.1 Diagramma della FSM

Il componente è stato realizzato come una **Macchina a Stati Finiti (FSM) di Mealy a 16 stati**, è stata una decisione di chiarezza e non di minimizzazione del numero di stati, di seguito viene illustrata la FSM nella figura 2.

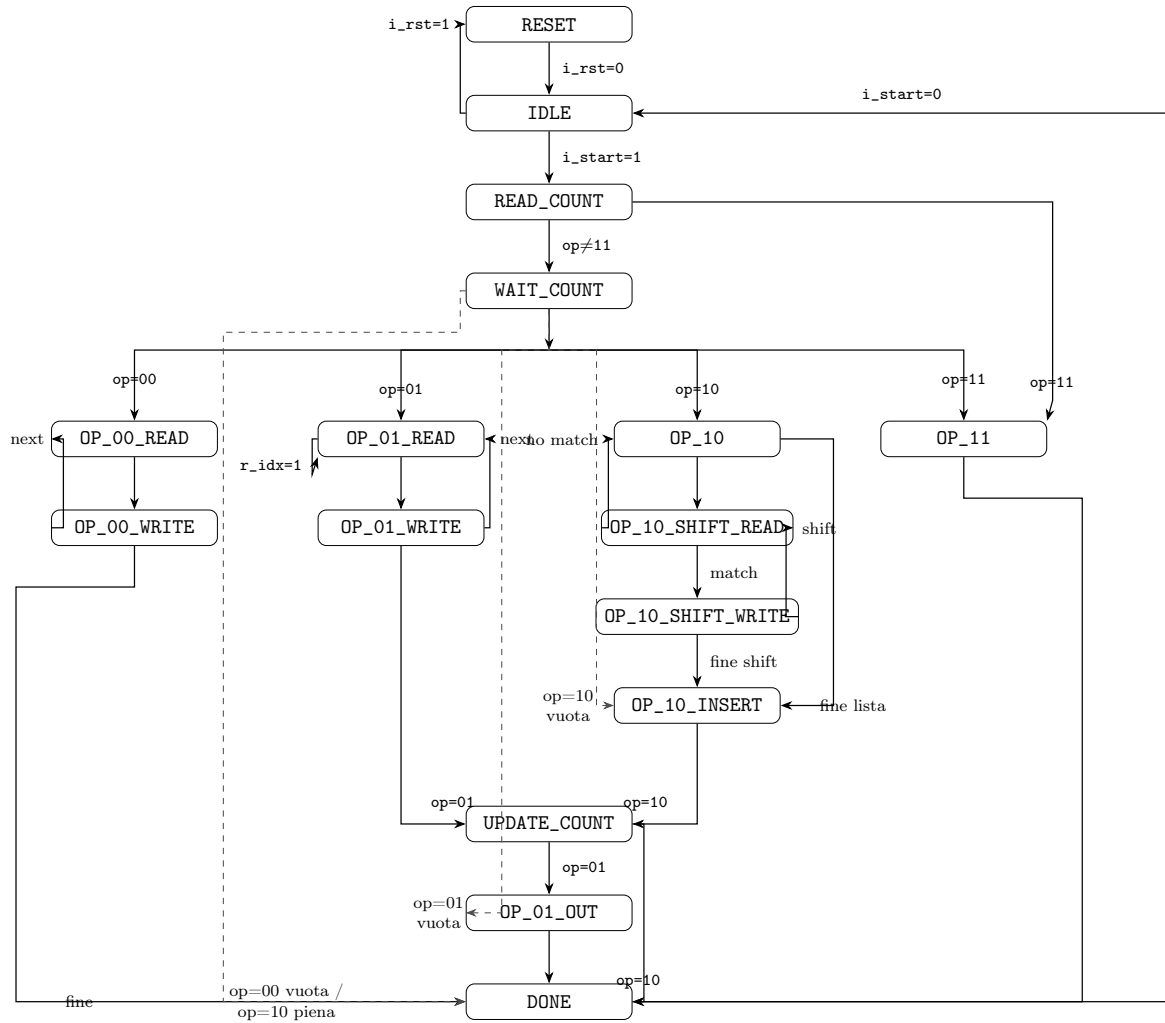


Figura 2: Diagramma completo della Macchina a Stati Finiti (FSM)

2.2 Descrizione degli stati

2.2 RESET

Viene attivato quando `i_rst='1'` in modo asincrono. Viene scritto 0 all'indirizzo 0 della RAM, azzerando il contatore dei task come da specifica, e `o_done` viene portato a '1' per segnalare che il modulo non è ancora pronto e, al ciclo successivo, si transisce in IDLE.

2.3 IDLE

È lo stato di attesa, dove non viene eseguita nessuna operazione finché `i_start` sale a 1, come da protocollo. Quando arriva il segnale, l'operazione richiesta viene salvata in `r_op` tramite `next_op` per evitare eventuali errori dovuti alle variazioni di `i_op` durante l'esecuzione, e si transisce a `READ_COUNT`.

2.4 READ_COUNT

Viene abilitata la lettura della RAM all'indirizzo 0 per leggere il contatore dei task (per default è già `addr=0`). Se `i_op=11` non serve sapere quanti task ci sono e, senza aspettare il dato, si transisce direttamente a `OP_11`. Per tutte le altre operazioni si transisce in `WAIT_COUNT`.

2.5 WAIT_COUNT

Il dato letto dalla RAM è adesso disponibile in `i_mem_data` (a questo punto contiene il valore di `addr=0`, ossia il numero di task) e viene salvato in `next_count`. Da qui la FSM gestisce i casi limite e transisce ai vari stati operazionali:

- `op=00`: se lista vuota transisce a `DONE`, altrimenti a `OP_00_READ`;
- `op=01`: se lista vuota transisce a `OP_01_OUT` con `r_first=0`, altrimenti a `OP_01_READ`;
- `op=10`: se lista vuota transisce a `OP_10_INSERT` con `r_insert_pos=1`, se è piena transisce a `DONE`, altrimenti transisce a `OP_10`.

2.6 OP_11

Viene scritto 0 all'indirizzo 0 della RAM, azzerando il contatore, come da specifica, il contenuto degli altri indirizzi rimane invariato, ma ignorato. Si transisce poi in `DONE`.

2.7 OP_00_READ

L'indirizzo `r_index` viene presentato alla RAM in lettura. Si transisce poi in `OP_00_WRITE`, dove il dato sarà disponibile per la scrittura e modifica.

2.8 OP_00_WRITE

Il dato arrivato da `i_mem_data` viene letto e riscritto all'indirizzo `r_index` incrementando di 1 i 2 bit di priorità del task, con saturazione a "11", come da specifica. L'operazione termina quando `r_index ≥ r_task_count` (si transisce in `DONE`), altrimenti `r_index` viene incrementato e si torna a `OP_00_READ`, creando un ciclo analogo al `for` del C.

2.9 OP_01_READ

La lista viene scorsa per preparare lo shift. Il primo ciclo con `r_index=1` serve esclusivamente a presentare l'indirizzo 1 alla RAM (dato non ancora pronto); l'indice viene poi incrementato e si rimane in `OP_01_READ`. Dal secondo ciclo con `r_index=2` il dato di `addr=1` è disponibile in `i_mem_data` e viene salvato in `r_first` (ID del task da rimuovere). Si transisce poi in `OP_01_WRITE`.

2.10 OP_01_WRITE

Il dato letto nel ciclo precedente viene scritto all'indirizzo `r_index - 1`, shiftando i task di una posizione verso sinistra. L'operazione termina quando `r_index = r_task_count + 1` (poiché `r_index` parte da 2) e si transisce in `UPDATE_COUNT`; altrimenti `r_index` viene incrementato e si torna a `OP_01_READ`.

2.11 OP_01_OUT

Viene emesso su `o_task_id` l'ID del primo task rimosso (salvato in `r_first`), oppure 0 se la lista era vuota, come da specifica. `o_done` viene portato a '1': questo è necessario in quanto la specifica richiede che `o_task_id` sia valido esattamente nel ciclo in cui `o_done=1`. Si transisce poi a `DONE`.

2.12 OP_10

L'indirizzo `r_index` viene presentato alla RAM. Se `r_index > r_task_count`, la posizione di inserimento è in fondo alla lista: viene impostato `r_insert_pos = r_task_count + 1` e si transisce in `OP_10_INSERT`. Altrimenti si transisce in `OP_10_SHIFT_READ`.

2.13 OP_10_SHIFT_READ

Questo stato ha due ruoli distinti in base a `r_insert_pos`: se `r_insert_pos > 0`, la posizione è già stata trovata e si transisce a `OP_10_SHIFT_WRITE`; se invece `r_insert_pos = 0`, viene confrontata la priorità del task letto (`i_mem_data[1:0]`) con quella del nuovo task tramite un confronto numerico esplicito con cast `unsigned()`, necessario perché `IEEE.NUMERIC_STD` non definisce il confronto `>` su `std_logic_vector` in sintesi. Se il task esistente ha priorità numericamente maggiore (gerarchicamente inferiore), `r_index` viene salvato come `r_insert_pos` e portato all'ultimo task per iniziare lo shift da destra; altrimenti `r_index` viene incrementato e si torna a `OP_10`, per continuare la ricerca.

2.14 OP_10_SHIFT_WRITE

Il dato letto nel ciclo precedente viene scritto in `r_index + 1`, shiftando ogni task di una posizione avanti. Quando `r_index = r_insert_pos`, ossia arrivati alla posizione di inserimento, si transisce in `OP_10_INSERT`; altrimenti `r_index` viene decrementato e si torna a `OP_10_SHIFT_READ`.

2.15 OP_10_INSERT

Il nuovo task viene scritto all'indirizzo `r_insert_pos`, combinando `i_task_id` e `i_task_priority` in un unico byte (`i_task_id & i_task_priority`). Si transisce poi in `UPDATE_COUNT`.

2.16 UPDATE_COUNT

Il contatore all'indirizzo 0 della RAM viene aggiornato in base all'operazione:

- OP_10: viene scritto `r_task_count + 1` e si transisce in DONE;
- OP_01: viene scritto `r_task_count - 1` e si transisce in OP_01_OUT per emettere l'ID del task rimosso.

2.17 DONE

È lo stato che rappresenta la fine dell'operazione e come da protocollo `o_done` viene portato a '1'. Si rimane in questo stato finché `i_start` non torna a 0 si transisce poi in IDLE.

3 Risultati Sperimentali

Per verificare il corretto funzionamento del componente sintetizzato, sono stati creati dei testbench ritenuti esaustivi nel testare il comportamento del componente. In particolare sono stati testati sia il funzionamento di tutte e quattro le operazioni implementate che i casi limite di queste ultime. Inoltre è stato verificato che il reset avvenga correttamente, con l'inizializzazione corretta dei registri.

3.1 Simulazioni Behavioral

Testbench 1

Caso: inserimento di vari task in momenti diversi, a seguito di reset a metà operazione, rimozione di alcuni task e infine svuotamento della lista. Questo testbench è stato progettato per verificare la robustezza del modulo in condizioni di utilizzo realistico e in presenza di un reset asincrono durante un'operazione in corso.

```
Log: Failure:  Simulation Ended!  TUTTI I TEST PASSATI -- Time:  2080 ns --  
Iteration:0
```

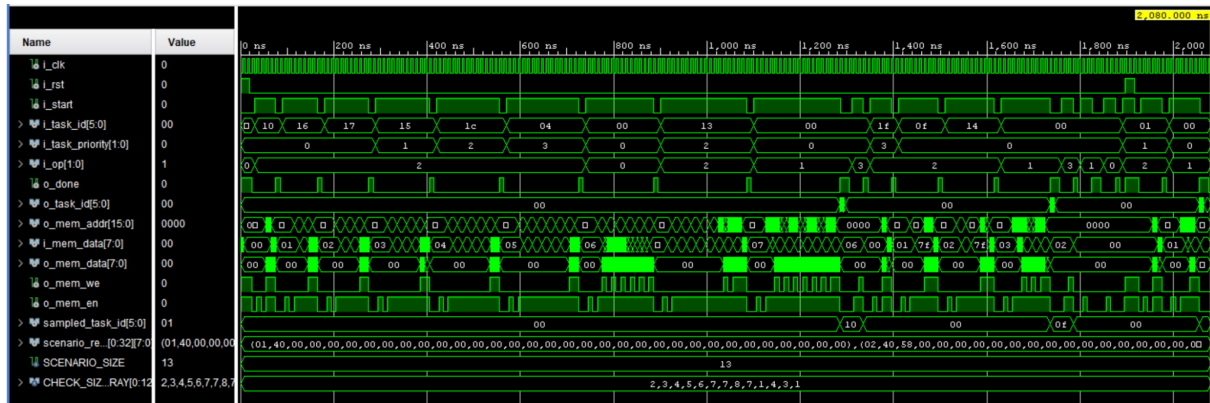


Figura 3: Waveform — Testbench 1 (Behavioral)

Testbench 2

Caso: reset, inserimento di 3 task, svuotamento lista. Testbench di base per verificare il corretto funzionamento del flusso reset → insert → OP=11.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 340 ns -- Iteration:0

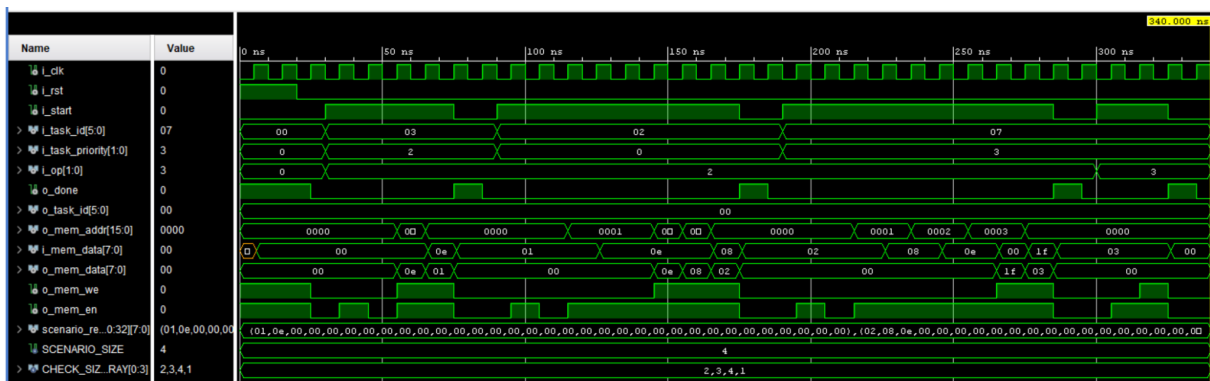


Figura 4: Waveform — Testbench 2 (Behavioral)

Testbench 3

Caso: inserimento di più task con priorità sparsa. Verifica che l'ordinamento della lista venga mantenuto correttamente al variare delle priorità dei task inseriti.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 760 ns -- Iteration:0

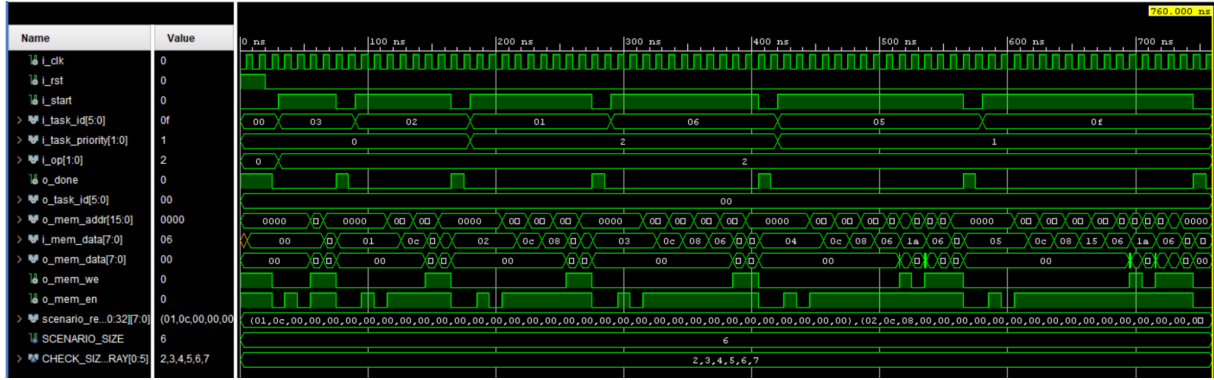


Figura 5: Waveform — Testbench 3 (Behavioral)

Testbench 4

Caso: inserimento di più task in ordine casuale (nessun task con priorità 0 inserito), seguito da $OP = 00$. Verifica che l'operazione di decremento della priorità funzioni correttamente anche in assenza di task con priorità massima.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 920 ns -- Iteration:0

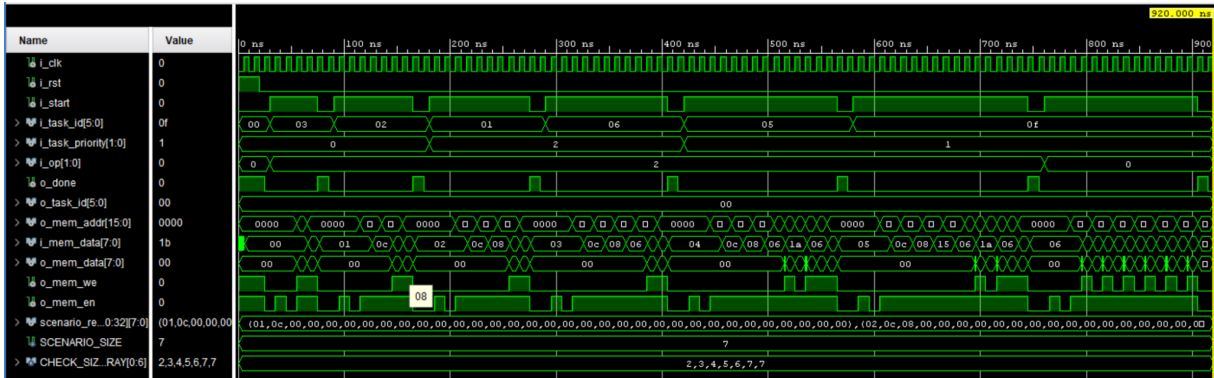


Figura 6: Waveform — Testbench 4 (Behavioral)

Testbench 5

Caso limite: $OP = 01$ su lista vuota. Verifica che l'uscita o_task_id sia 0 e che il modulo non vada in stallo.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 80 ns --- Iteration:0

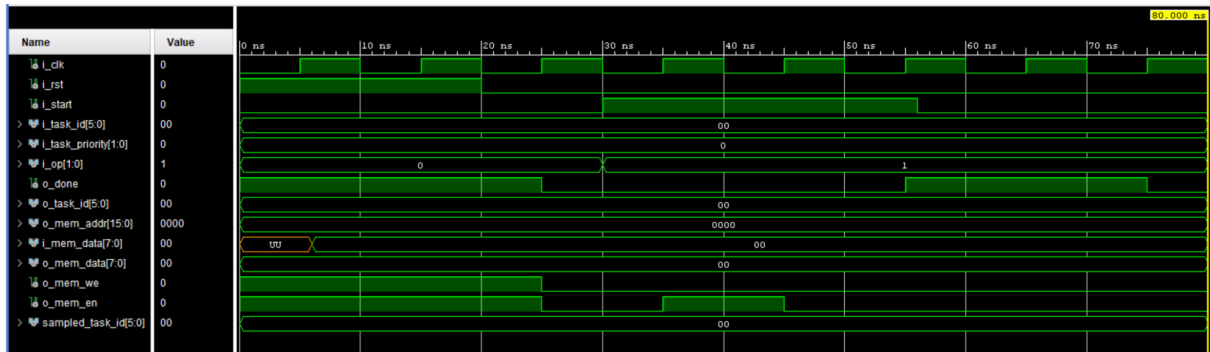


Figura 7: Waveform — Testbench 5 (Behavioral) — Lista vuota con OP=01

Testbench 6

Caso: inserimento di task con PRIORITY = "11", seguito da OP = 00. Verifica la saturazione: i task già a priorità "11" devono rimanere invariati.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 420 ns -- Iteration:0

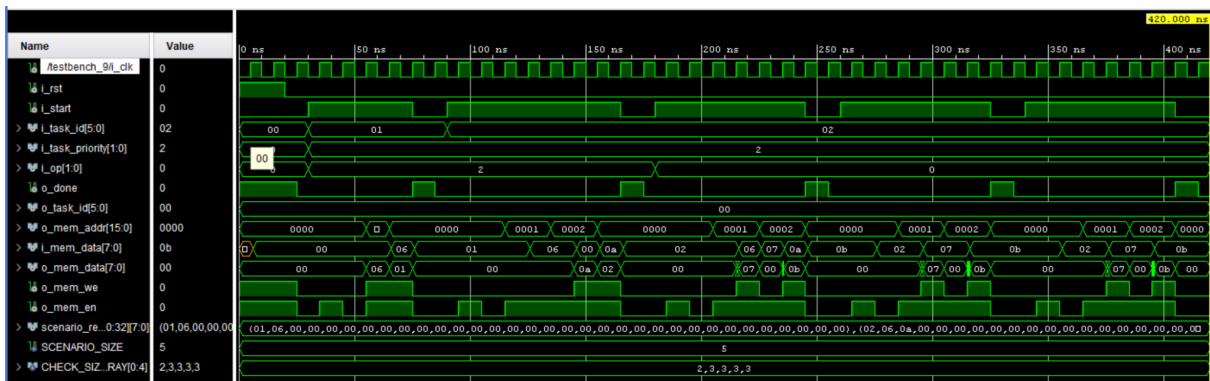


Figura 8: Waveform — Testbench 6 (Behavioral) — Saturazione priorità

Testbench 7

Caso limite: inserimento di 63 task (lista piena) e tentativo di inserimento del 64° task. Verifica che il modulo ignori correttamente l'inserimento in eccesso.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 43840 ns -- Iteration:0

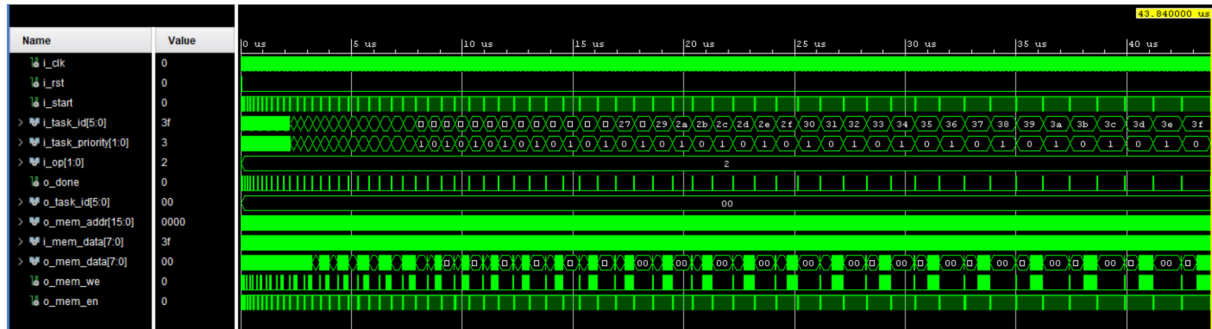
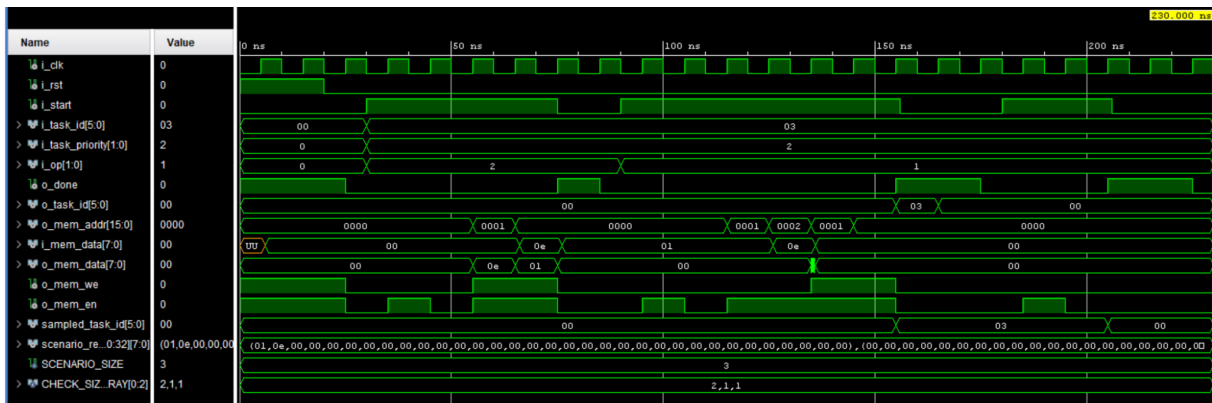


Figura 9: Waveform — Testbench 7 (Behavioral) — Lista piena (63 task)

Testbench 8

Caso: inserimento di un singolo task, seguito da $OP = 01$. Verifica la corretta rimozione e restituzione dell'ID in una lista con un unico elemento.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 230 ns --
Iteration:0

Figura 10: Waveform — Testbench 8 (Behavioral) — Singolo task con $OP=01$

Testbench 9

Caso: inserimento di più task, $OP = 01$ ripetuto in sequenza fino a svuotare completamente la lista. Verifica la correttezza dello shift e del contatore ad ogni estrazione.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 1300 ns --
Iteration:0

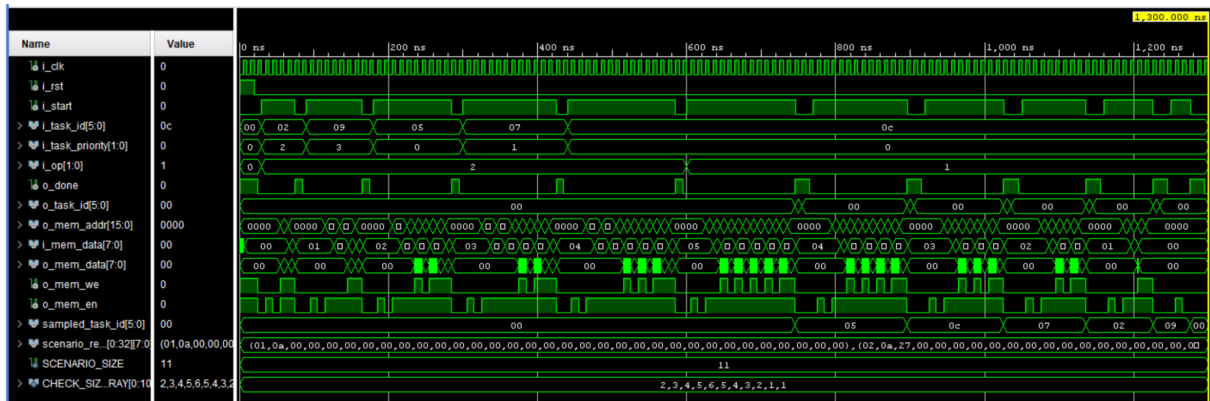


Figura 11: Waveform — Testbench 9 (Behavioral) — Svuotamento con OP=01 ripetuto

Testbench 10

Caso limite: OP = 11 su lista già vuota. Verifica che l'operazione di svuotamento si comporti correttamente anche quando il contatore è già 0.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 360 ns --
Iteration:0

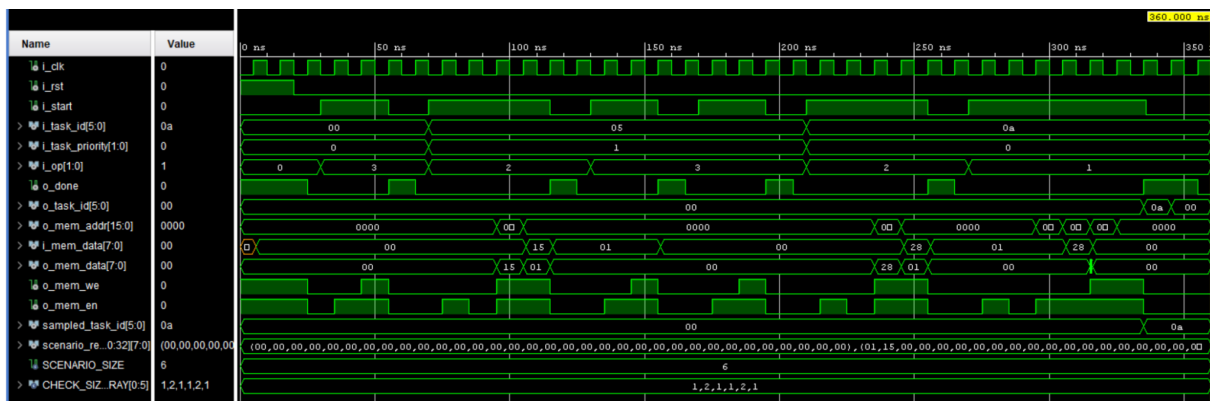


Figura 12: Waveform — Testbench 10 (Behavioral) — OP=11 su lista vuota

Testbench 11

Caso: inserimento di 63 task (lista piena), seguito da OP = 00. Verifica la saturazione su tutta la lista nel caso peggiore di dimensione massima.

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 47840 ns --
Iteration:0

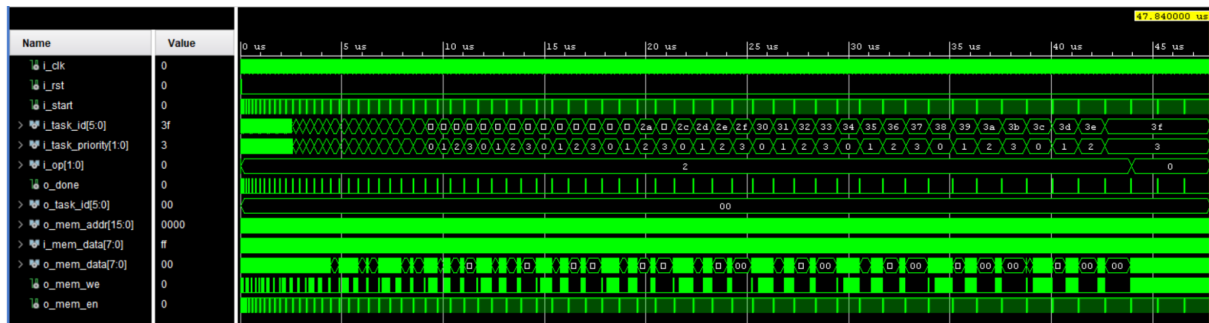


Figura 13: Waveform — Testbench 11 (Behavioral) — OP=00 su lista piena

Testbench 12

Caso: inserimento di due task con priorità diversa per verificare il corretto posizionamento: il primo ha priorità numericamente più bassa (gerarchicamente più alta), il secondo più alta. Verifica che il secondo task venga inserito nella posizione corretta rispetto all'ordinamento.

```
Log: Failure:  Simulation Ended!  TUTTI I TEST PASSATI -- Time:  750 ns --
Iteration:0
```

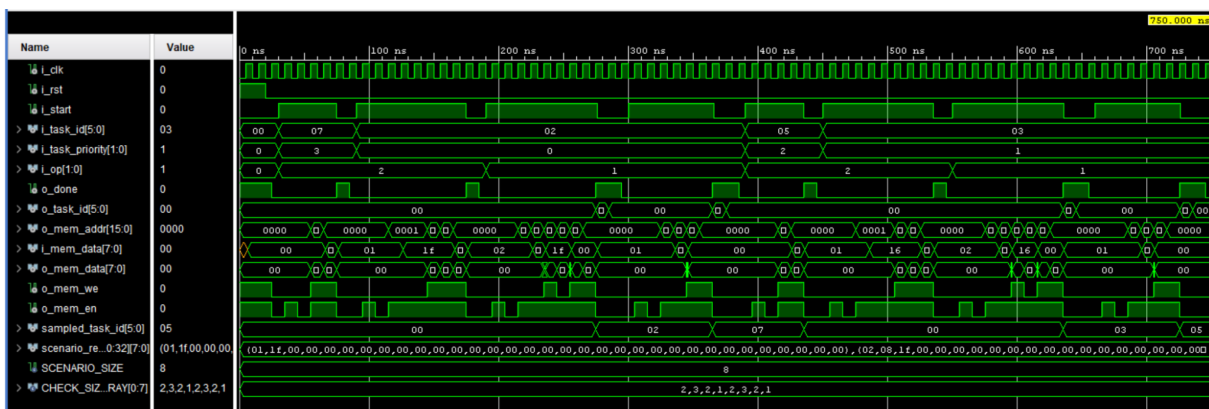


Figura 14: Waveform — Testbench 12 (Behavioral) — Inserimento in ordine di priorità

Testbench 13 — Testbench della specifica

Caso: testbench ufficiale fornito dalla specifica del corso.

```
Log: Failure:  Simulation Ended!  TEST PASSATO (EXAMPLE) -- Time:  1880 ns
-- Iteration:0
```

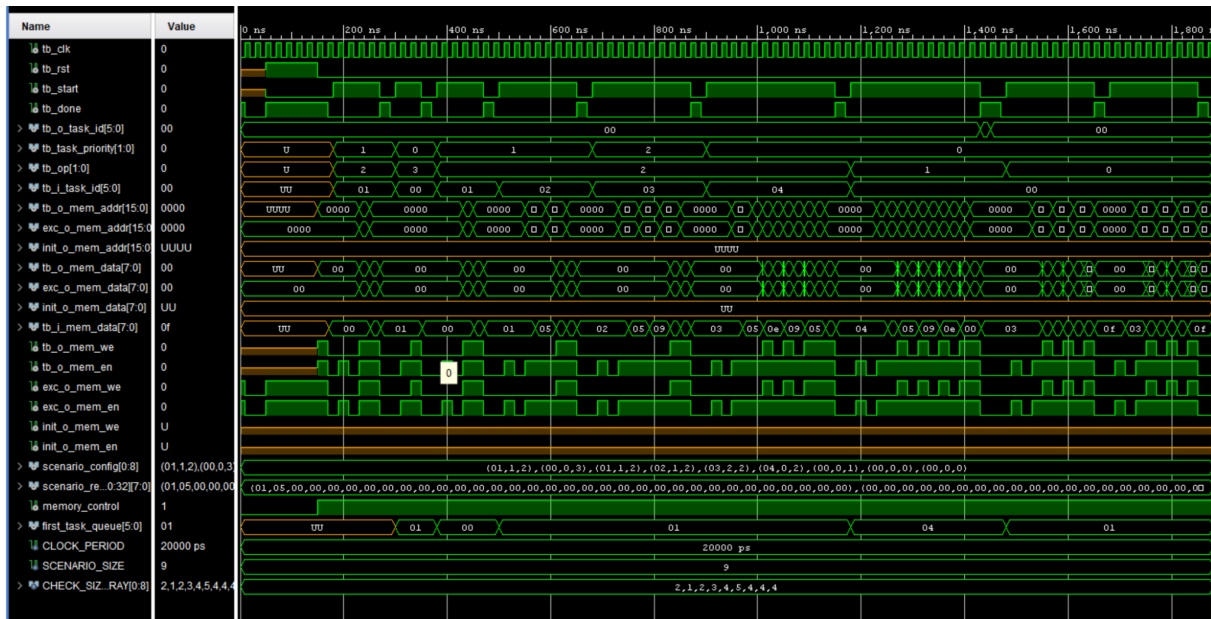


Figura 15: Waveform — Testbench 13 (Testbench della specifica)

3.2 Simulazioni Post-Synthesis Functional

Il comportamento nelle simulazioni *Post-Synthesis Functional* è uguale a quello nelle simulazioni *Behavioral* e supera il testbench d'esempio fornito. Utilizzando, ad esempio, il Testbench 1, il test viene superato:

Log: Failure: Simulation Ended! TUTTI I TEST PASSATI -- Time: 2080 ns - Iteration:0

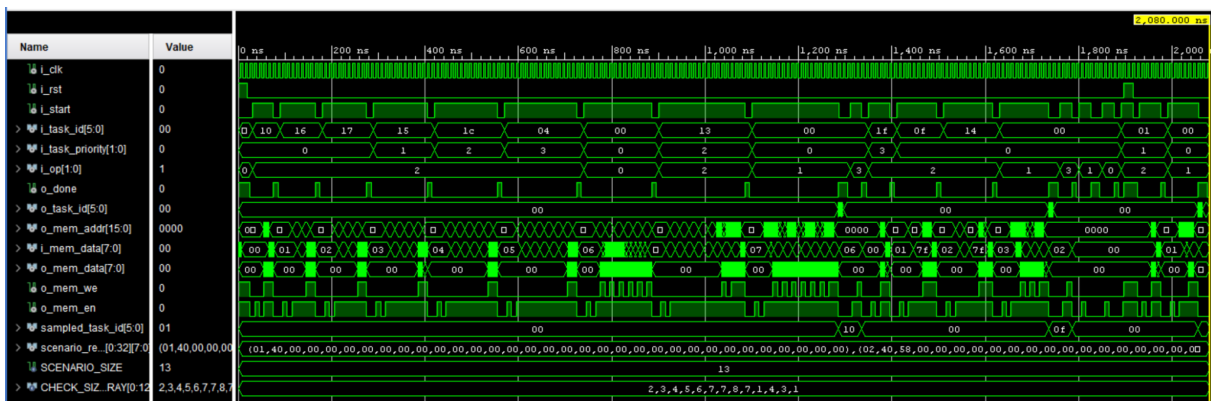


Figura 16: Waveform — Testbench 1 (Post-Synthesis Functional)

3.3 Report di Sintesi

Timing

Il report di timing conferma che il progetto soddisfa i requisiti imposti dalla specifica, con uno slack positivo di **14.639 ns** rispetto al periodo di clock di 20 ns.

Timing Report

Slack (MET) : 14.639ns (required time - arrival time)

Figura 17: Timing Report — Slack (MET): 14.639 ns

Utilization

Il report di utilization mostra che l'implementazione non presenta **alcun latch**, confermando la correttezza delle scelte progettuali e della logica di funzionamento.

Site Type	Used	Fixed	Available	Util%
Slice LUTs	135	0	133800	0.10
LUT as Logic	135	0	133800	0.10
LUT as Memory	0	0	46200	0.00
Slice Registers	30	0	267600	0.01
Register as Flip Flop	30	0	267600	0.01
Register as Latch	0	0	267600	0.00
F7 Muxes	2	0	66900	<0.01
F8 Muxes	0	0	33450	0.00

Figura 18: Utilization Report — 0 latch, 30 Flip-Flop, 135 LUT

4 Conclusione

La progettazione del modulo è avvenuta privilegiando la chiarezza e la correttezza del flusso di controllo, rispetto alla minimizzazione del numero di stati; innanzitutto, sono stati individuati i requisiti principali e i casi limite del progetto; solo successivamente, si è passati all'elaborazione della macchina a stati.

Per evitare duplicazioni di logica, si è deciso di centralizzare la lettura del contatore di task negli stati `READ_COUNT` e `WAIT_COUNT`, condivisa da tutte le operazioni ad eccezione di `op=11`, che viene instradata già in `READ_COUNT` prima che il dato dalla RAM sia disponibile. Inoltre, come decisione di efficienza, direttamente in `WAIT_COUNT`, i casi limite sono stati gestiti prima di transitare nei rami degli stati operativi.

La progettazione della FSM e dei testbench hanno messo in luce anche alcuni difetti di progettazione; in particolare, una delle sfide principali è stato la realizzazione degli stati che si occupano delle operazioni 10 e 01. In `op=10` è stato aggiunto un cast esplicito `unsigned()` al confronto tra priorità, poiché nella libreria `IEEE.NUMERIC_STD` il confronto `>` su `std_logic_vector` non è definito numericamente, producendo un comportamento non deterministico in sintesi. Invece, in `op=01` è stata migliorata la gestione del timing, per

risolvere dei problemi dovuto alla latenza della memoria sincrona, infatti in `OP_01_READ` il primo ciclo con `r_index=1` serve esclusivamente a presentare l'indirizzo alla RAM.

In conclusione, il modulo sintetizzato funziona correttamente con tutti i testbench eseguiti, sia in simulazione *Behavioral* che in *Post-Synthesis Functional*, determinando il successo del nostro progetto.